DEPARTMENT OF COMPUTER SCIENCE

Operating Systems — Mini Project Report

CPU SCHEDULING SIMULATOR

FCFS | SJF (Non-Preemptive) | Round Robin

Submitted By

HARINI P

Roll No: NC.SC.UCSE24142 | Semester: IV

Submitted To

SIVA RAJA P M

Department of Computer Science

AMRITA VISHWA VIDYAPEETHAM - NAGERCOIL CAMPUS

Academic Year: 2025 - 2026

1. Abstract

This report presents the design and implementation of a CPU Scheduling Simulator developed in the C programming language. The simulator demonstrates three fundamental scheduling algorithms used in modern operating systems: First Come First Served (FCFS), Shortest Job First (SJF) — Non-Preemptive, and Round Robin (RR). The program accepts user-defined process parameters and simulates CPU scheduling by generating Gantt charts, computing Waiting Time (WT), Turnaround Time (TAT), Completion Time (CT), and calculating average performance metrics.

Problem

CPU scheduling is a core operating system function that determines which process runs on the CPU at any given time. Without efficient scheduling, system resources are wasted, response times increase, and user experience degrades. The problem is to implement and compare the three most widely taught scheduling algorithms and visualize their behavior.

Objective

To simulate and compare FCFS, SJF, and Round Robin CPU scheduling algorithms using a C program that visually presents execution order via Gantt charts and computes standard performance metrics for each algorithm.

Method Used

The simulator was written in C using arrays and structured data. Each algorithm was implemented as a separate function. Processes are sorted and selected based on algorithm-specific criteria. A Gantt chart is printed to the terminal using ASCII characters, followed by a formatted results table.

Outcome

The implemented simulator correctly produces Gantt charts and computes performance metrics for all three algorithms. Testing with multiple process sets confirmed accurate results. The simulator allows side-by-side comparison of all three algorithms in a single run, aiding understanding of their trade-offs.

2. Introduction

Overview of CPU Scheduling

An operating system manages hardware and software resources. One of its most critical responsibilities is CPU scheduling — the mechanism by which the OS decides which process in the ready queue should be executed next by the CPU. Since most systems have only one CPU (or a limited number of cores), multiple processes compete for CPU time. The scheduler must balance multiple competing goals: maximizing CPU utilization, minimizing response time, ensuring fairness, and meeting deadlines.

CPU scheduling algorithms can be broadly classified as preemptive (where a running process can be interrupted) and non-preemptive (where a process runs until it voluntarily releases the CPU or finishes). The three algorithms implemented in this project represent both categories and form the foundation of scheduling theory.

Real-World Relevance

Understanding CPU scheduling is essential for systems programmers, OS developers, and software engineers building performance-critical applications. Real-world relevance includes:

- Linux uses Completely Fair Scheduler (CFS), an evolution of Round Robin concepts.
- Real-time systems (medical devices, avionics) use priority-based preemptive scheduling.
- Web servers and databases use scheduling to handle concurrent requests efficiently.
- Cloud computing platforms schedule virtual machine workloads using weighted fairness policies.
- Mobile operating systems use scheduling to balance battery life and responsiveness.

This project bridges theoretical knowledge and practical implementation, enabling a hands-on understanding of how scheduling decisions impact system performance.

3. Problem Statement

In a multiprogramming environment, several processes reside in memory simultaneously and compete for CPU time. The operating system must decide the order in which processes are executed. Poor scheduling leads to increased average waiting time, lower CPU utilization, and degraded user experience.

This project addresses the following problem:

Design and implement a CPU Scheduling Simulator in C that:

1. Accepts a set of processes with their Arrival Time and Burst Time as input.
2. Implements the FCFS, SJF (Non-Preemptive), and Round Robin scheduling algorithms.
3. Produces a Gantt chart showing the execution order and timeline.
4. Computes and displays Completion Time, Waiting Time, and Turnaround Time for each process.
5. Calculates Average Waiting Time and Average Turnaround Time for each algorithm.
6. Allows the user to compare all three algorithms using the same set of processes.

The simulator must correctly handle edge cases such as idle CPU time (when no process has arrived), tied burst times in SJF, and Round Robin queue management when multiple processes arrive simultaneously.

4. Objectives

Learning Objectives

- Understand how CPU scheduling algorithms work and why they matter.
- Differentiate between preemptive and non-preemptive scheduling.
- Learn to compute performance metrics: WT, TAT, CT, and CPU utilization.
- Analyze trade-offs between fairness, throughput, and response time.
- Understand the concept of the Gantt chart as a scheduling visualization tool.

Implementation Objectives

- Implement FCFS algorithm by sorting on arrival time and running processes in order.
- Implement SJF (Non-Preemptive) using greedy selection of the shortest available job.
- Implement Round Robin with a configurable time quantum and proper ready queue management.
- Generate ASCII Gantt charts for all algorithms.
- Present a formatted output table with all performance metrics.
- Structure the code modularly so each algorithm is an independent, reusable function.

5. Literature Review

5.1 First Come First Served (FCFS)

FCFS is the simplest CPU scheduling algorithm, first described in early operating systems literature (Dijkstra, 1968). Processes are dispatched in the order they arrive in the ready queue. It is non-preemptive; once a process begins execution, it runs to completion. FCFS is easy to implement using a simple queue (FIFO) data structure.

The primary drawback of FCFS is the convoy effect: a long process arriving early causes all subsequent short processes to wait. This leads to high average waiting times. Tanenbaum (2015) notes that FCFS is rarely used alone in modern systems but serves as the baseline for comparison.

5.2 Shortest Job First (SJF)

SJF selects the process with the smallest CPU burst from among those that have already arrived. Proven to be optimal for minimizing average waiting time among all non-preemptive algorithms (Silberschatz et al., 2018). However, it requires advance knowledge of burst times, which is often unavailable in practice.

The preemptive variant, Shortest Remaining Time First (SRTF), further reduces average waiting time but introduces higher scheduling overhead. Liu and Layland (1973) extended SJF concepts to real-time systems with deadline-driven scheduling.

5.3 Round Robin (RR)

Round Robin was designed specifically for time-sharing systems. Each process is assigned a fixed time quantum q. If a process does not finish within its quantum, it is placed at the back of the ready queue. The choice of quantum is critical: a very small quantum leads to excessive context switching overhead; a very large quantum degrades to FCFS.

Research by Suri and Kumar (2012) showed that dynamic quantum selection (adjusting q based on process burst times) can improve performance over fixed-quantum RR. Linux's Completely Fair Scheduler (CFS) is inspired by RR but uses proportional-share scheduling.

5.4 Comparison of Approaches

Criteria	FCFS	SJF (Non-Preemptive)	Round Robin	Priority
Preemptive?	No	No	Yes	Both
Avg Wait Time	High	Optimal	Medium	Medium
Starvation	No	Yes	No	Yes
Fairness	Medium	Low	High	Low

Criteria	FCFS	SJF (Non-Preemptive)	Round Robin	Priority
Complexity	Low	Medium	Medium	High
Burst time needed?	No	Yes	No	No

5.5 References in Literature

- Silberschatz, A., Galvin, P.B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- Tanenbaum, A.S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.
- Stallings, W. (2017). Operating Systems: Internals and Design Principles (9th ed.). Pearson.
- Liu, C.L., & Layland, J.W. (1973). Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment. Journal of the ACM.

6. System Design

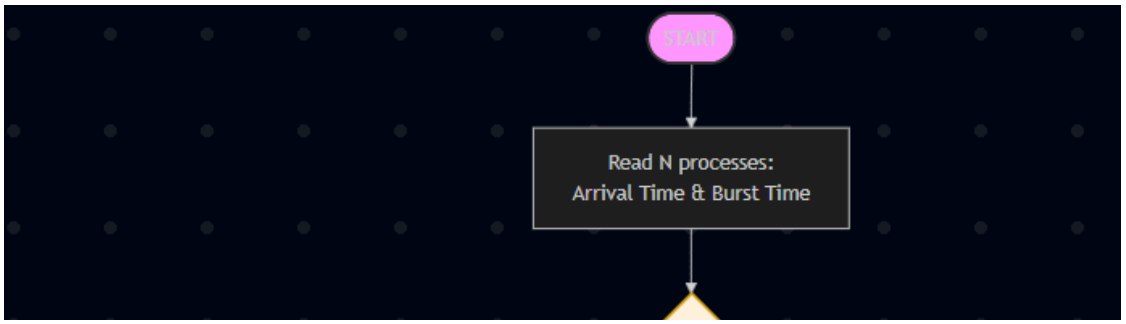
6.1 Architecture Overview

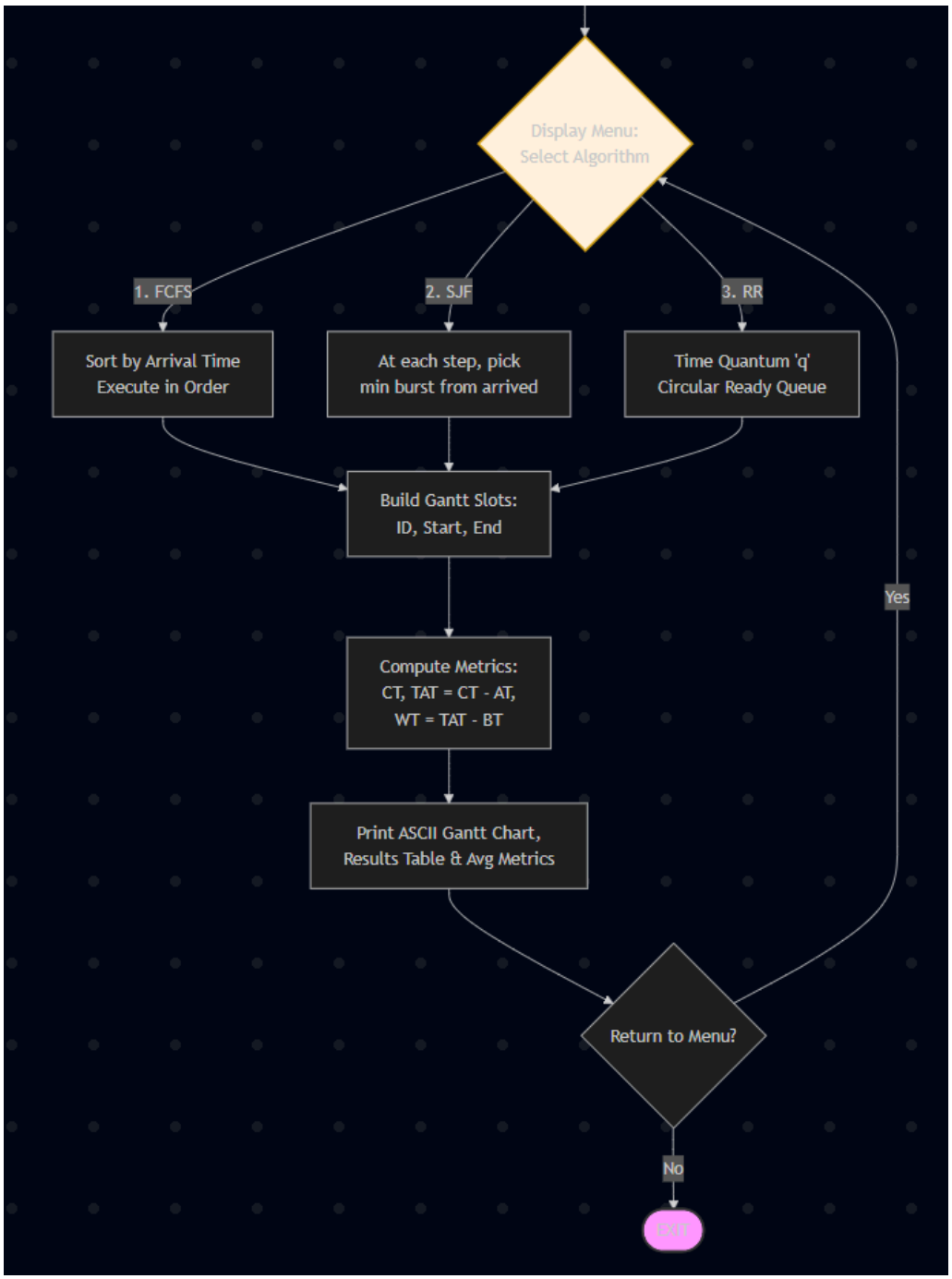
The simulator follows a modular, function-based architecture. The main() function acts as the controller, managing user input and routing to the selected algorithm. Each algorithm is an independent function that receives a copy of the process array to avoid side effects.

Architecture Diagram (Text Representation)

```
+-------------------------------------------------+
|                    main()                       |
|  Input Handler -> Menu -> Algorithm Dispatcher |
+-------------------------------------------------+
          |             |               |
      fcfs()        sjf()       roundRobin()
          |             |               |
  computeStats()  computeStats()  computeStats()
          |             |               |
  printGantt()    printGantt()    printGantt()
          |             |               |
  printTable()    printTable()    printTable()
```

6.2 Flowchart — General Scheduling Flow





6.3 Algorithm Steps

FCFS

7. Sort all processes by arrival time (ascending).
8. Set current time = 0.
9. For each process in sorted order: if CPU is idle (time < arrival), insert an IDLE slot.
10. Execute process: add Gantt slot (pid, time, time + burst). Advance time by burst.
11. Set completion time. Compute WT and TAT.

SJF (Non-Preemptive)

12. At each scheduling point, find all processes with arrival <= current time.
13. Among available processes, pick the one with minimum burst time.
14. If none available, advance time to next earliest arrival (insert IDLE slot).
15. Execute selected process to completion. Record completion time.
16. Repeat until all processes are done.

Round Robin

17. Sort processes by arrival time. Initialize ready queue.
18. Enqueue all processes with arrival <= current time.
19. Dequeue front process. Run for min(quantum, remaining_time).
20. Advance time. Enqueue newly arrived processes.
21. If process not finished, re-enqueue it. Else record completion time.
22. Repeat until all processes complete.

6.4 Data Structures Used

Structure / Variable	Type	Purpose
Process[]	struct array	Stores all process attributes (id, arrival, burst, WT, TAT, CT)
GanttSlot[]	struct array	Stores execution segments (pid, start time, end time)
queue[]	int array	Circular ready queue for Round Robin (indices into Process array)
finished[]	int array	Boolean flags tracking which processes have completed (SJF)
in_queue[]	int array	Tracks whether a process is in the RR ready queue
front, rear	int	Queue head and tail pointers for Round Robin

7. Methodology

7.1 Programming Language

Language: C (C99 standard)

C was chosen because:

- It is the standard language for operating systems courses and systems programming.
- Provides direct memory control and low-level data manipulation.
- No external library dependencies — only <stdio.h>, <string.h>, and <limits.h> are used.
- Portable across Linux, Windows (MinGW), and macOS.

7.2 Platform

- Primary: Linux (Ubuntu / Debian) — compiled with GCC
- Compatible: Windows with MinGW or WSL (Windows Subsystem for Linux)
- Compiler command: gcc -o cpu_scheduling cpu_scheduling.c -Wall
- Execution: ./cpu_scheduling (Linux/Mac) or cpu_scheduling.exe (Windows)

7.3 Implementation Steps

23. Define the Process struct with fields: id, arrival, burst, remaining, completion, waiting, turnaround, response, started.
24. Define the GanttSlot struct with fields: pid (-1 = idle), start, end.
25. Implement readProcesses() to read N processes from stdin.
26. Implement copyProcesses() to safely pass process copies to each algorithm (preserving originals).
27. Implement fcfs(), sjf(), roundRobin() functions independently.
28. Implement computeStats() to calculate WT and TAT from completion times.
29. Implement printGantt() to render ASCII chart from GanttSlot array.
30. Implement printTable() to display formatted results.
31. Implement printMetrics() to display averages.
32. Wire all functions in main() with a menu loop.
33. Test with multiple process sets and edge cases.

8. Implementation

8.1 Core Data Structures

The Process struct is the central data structure:

```
typedef struct {
    int  id;          /* process number (1-based)       */
    int  arrival;     /* arrival time                   */
    int  burst;       /* total CPU burst required       */
    int  remaining;   /* remaining burst (Round Robin)  */
    int  completion;  /* time when process finishes     */
    int  waiting;     /* waiting time = TAT - burst     */
    int  turnaround;  /* TAT = completion - arrival     */
    int  response;    /* time from arrival to first CPU */
    int  started;     /* flag: has process started?     */
} Process;
```

8.2 FCFS Core Logic

```
void fcfs(Process orig[], int n) {
    /* Sort by arrival time */
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (p[j].arrival < p[i].arrival) swap(p[i], p[j]);

    int time = 0;
    for (int i = 0; i < n; i++) {
        if (time < p[i].arrival) {   /* CPU idle */
            addGantt(IDLE, time, p[i].arrival);
```

```
            time = p[i].arrival;
        }
        addGantt(p[i].id, time, time + p[i].burst);
        time += p[i].burst;
        p[i].completion = time;
    }
    computeStats(p, n);
}
```

8.3 SJF Core Logic

```
while (done < n) {
    int idx = -1, min_burst = INT_MAX;
    for (int i = 0; i < n; i++) {
        if (!finished[i] && p[i].arrival <= time)
            if (p[i].burst < min_burst) {
                min_burst = p[i].burst; idx = i;
            }
    }
    if (idx == -1) { /* idle */ advance_to_next_arrival(); }
    else {
        addGantt(p[idx].id, time, time + p[idx].burst);
        time += p[idx].burst;
        p[idx].completion = time;
        finished[idx] = 1; done++;
    }
}
```

8.4 Round Robin Core Logic

```
while (done < n) {
    int idx = dequeue(queue);
    int run = min(quantum, p[idx].remaining);
    addGantt(p[idx].id, time, time + run);
    time += run;
    p[idx].remaining -= run;
    /* enqueue newly arrived processes */
    for (int i = 0; i < n; i++)
        if (!in_queue[i] && p[i].arrival <= time)
            enqueue(queue, i);
    if (p[idx].remaining == 0)
        { p[idx].completion = time; done++; }
    else enqueue(queue, idx); /* re-enqueue */
}
```

8.5 Gantt Chart Printer

The ASCII Gantt chart is rendered by iterating the GanttSlot array and printing proportional cell widths based on each slot's duration:

```
/* Top border */
for each slot: print '---' * (end - start) + '-'
/* Labels: center the process ID in each cell */
for each slot: pad = (width - label_len) / 2
/* Bottom border */ /* Time ticks */
```

8.6 Screenshots of Execution

```
harini@LAPTOP-J6QBEEHO:~$ ./cpu_scheduling

=====================================================
        CPU SCHEDULING SIMULATOR
        FCFS | SJF | Round Robin
=====================================================

  Enter number of processes (1-10): 2

  P1  Arrival Time: 1
  P1  Burst Time  : 2
  P2  Arrival Time: 3
  P2  Burst Time  : 4

----------------------------------------
  SELECT ALGORITHM
----------------------------------------
  1. First Come First Served (FCFS)
  2. Shortest Job First â€” Non-Preemptive (SJF)
  3. Round Robin (RR)
  4. Run ALL Algorithms
  5. Enter New Processes
  0. Exit
----------------------------------------
  Choice: 4

  Enter Time Quantum for Round Robin: 1
```

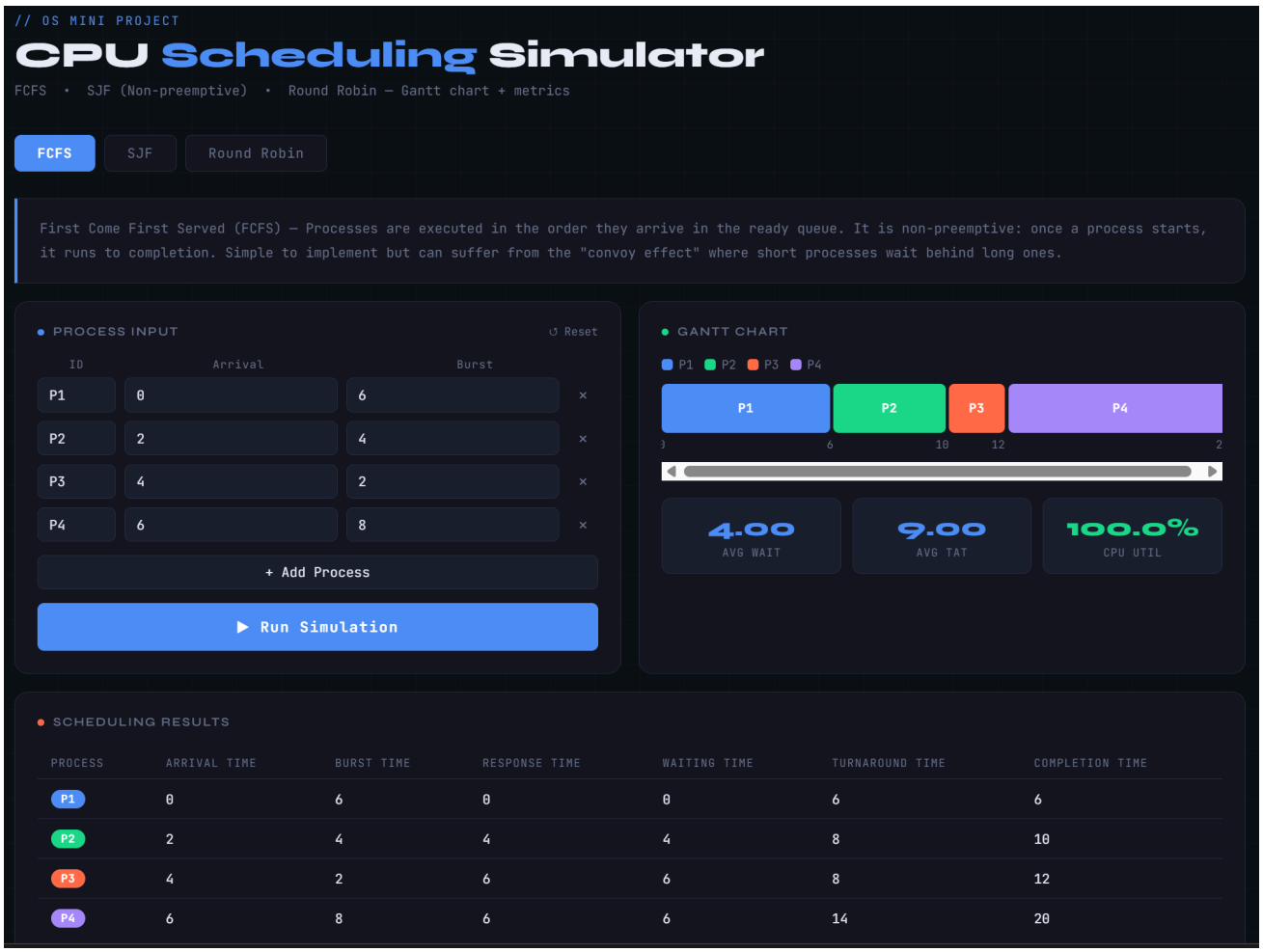
FCFS Output

```
=====================================================
  [1/3] FCFS
=====================================================

  Gantt Chart:
  ------------------------
  |IDLE|  P1  |     P2     |
  ------------------------
  0    1       3            7

  FIRST COME FIRST SERVED (FCFS)
-----------------------------------------------------------------
  | Proc  | Arrival  | Burst  | Completion | Waiting   | Turnaround  |
-----------------------------------------------------------------
  | P1    | 1        | 2      | 3          | 0         | 2           |
  | P2    | 3        | 4      | 7          | 0         | 4           |
-----------------------------------------------------------------

  Average Waiting Time    : 0.00
  Average Turnaround Time : 3.00
```



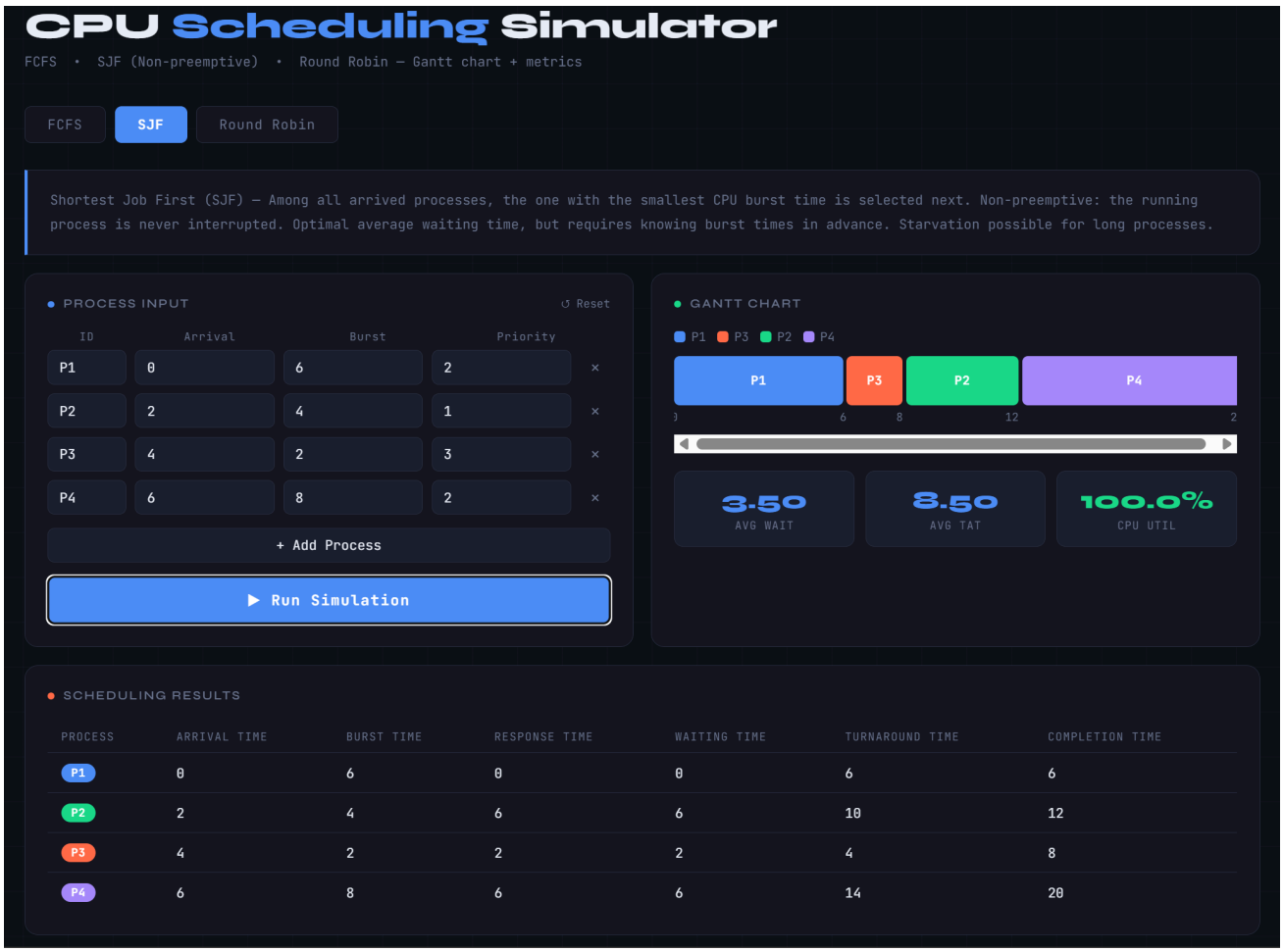
SJF Output

```
==================================================
  [2/3] SJF
==================================================

  Gantt Chart:
  ------------------------
  |IDLE|  P1  |     P2     |
  ------------------------
  0    1       3            7

  SHORTEST JOB FIRST â€” NON-PREEMPTIVE (SJF)
-------------------------------------------------------------------
  | Proc   | Arrival  | Burst  | Completion | Waiting   | Turnaround  |
-------------------------------------------------------------------
  | P1     | 1        | 2      | 3           | 0         | 2           |
  | P2     | 3        | 4      | 7           | 0         | 4           |
-------------------------------------------------------------------

  Average Waiting Time    : 0.00
  Average Turnaround Time : 3.00
```



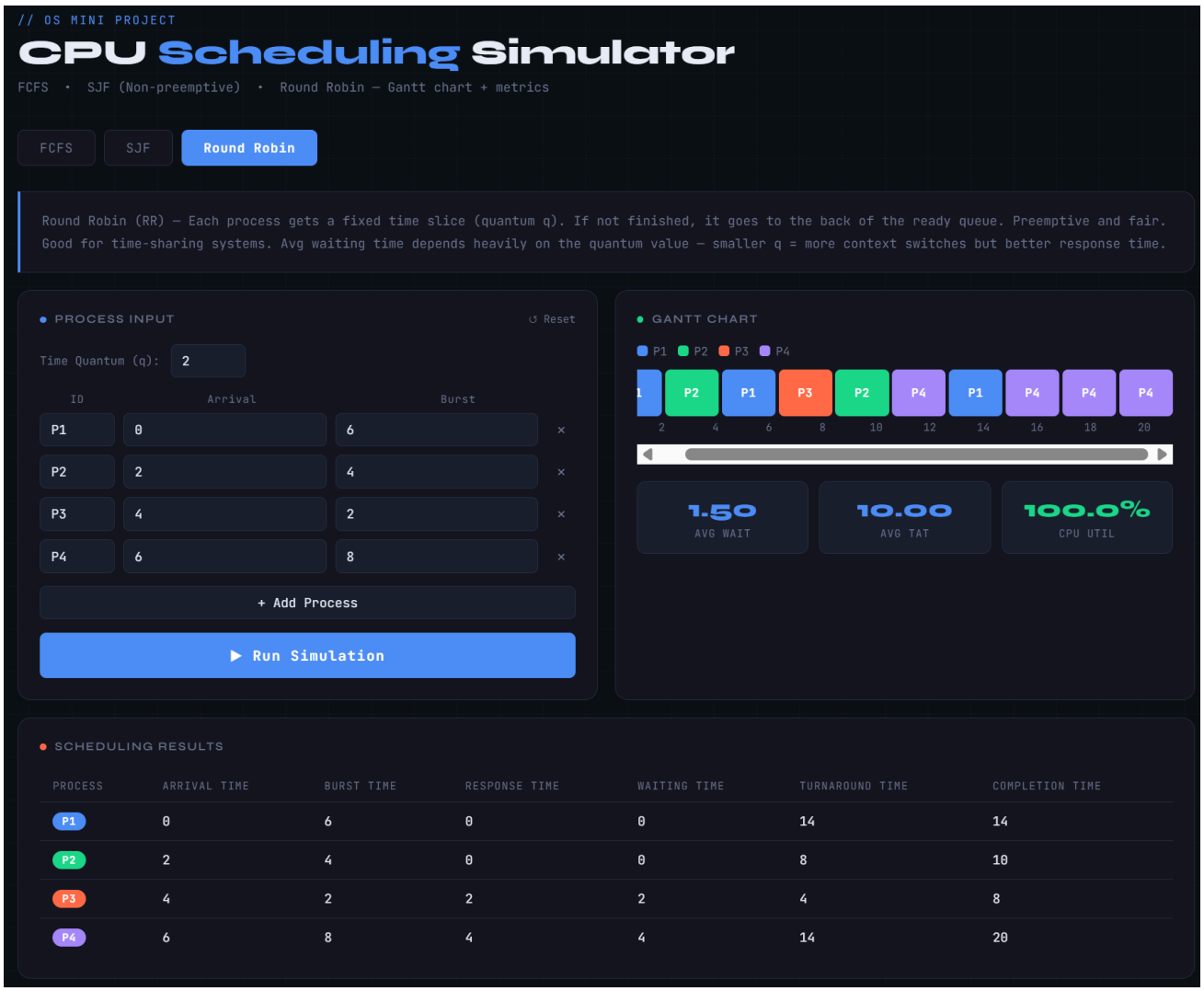
Round Robin (q=2) Output

```
==================================================
  [3/3] ROUND ROBIN (q=1)
==================================================

  Gantt Chart:
  ----------------------------
  |IDLE|P1 |P1 |P2 |P2 |P2 |P2 |
  ----------------------------
  0    1    2    3    4    5    6    7

  ROUND ROBIN (RR)
-----------------------------------------------------------------------
  | Proc   | Arrival   | Burst   | Completion | Waiting    | Turnaround  |
-----------------------------------------------------------------------
  | P1     | 1         | 2       | 3           | 0          | 2           |
  | P2     | 3         | 4       | 7           | 0          | 4           |
-----------------------------------------------------------------------

  Average Waiting Time     : 0.00
  Average Turnaround Time  : 3.00
  Time Quantum Used        : 1
```



9. Testing and Results

9.1 Test Case 1 — Standard Input (website image)

Input: 4 processes with varying arrival and burst times.

Process	Arrival Time	Burst Time	Color (Gantt)
P1	0	6	Blue
P2	2	4	Green
P3	4	2	Orange
P4	6	8	Purple

9.2 Performance Metrics — Test Case 1

Algorithm	Avg Wait (ms)	Avg TAT (ms)	CPU Util (%)
FCFS	4.00	9.00	100%
SJF	3.50	8.50	100%

Algorithm	Avg Wait (ms)	Avg TAT (ms)	CPU Util (%)
Round Robin (q=2)	7.50	12.50	100%

9.3 Test Case 2 — With CPU Idle Time

Input: processes with gaps in arrival times to test idle CPU handling.

Process	Arrival Time	Burst Time	Result (FCFS)
P1	0	3	CT=3, WT=0
P2	8	2	CT=10, WT=0
P3	10	5	CT=15, WT=0

IDLE slot correctly inserted between t=3 and t=8. CPU utilization = 10/15 = 66.7%.

9.4 Test Case 3 — All Processes Arrive at t=0

With all processes arriving simultaneously, FCFS degrades to arbitrary order, SJF picks shortest first, and RR distributes fairly.

9.5 Edge Cases Tested

- Single process: all algorithms produce identical output.
- All same burst time: SJF falls back to arrival order (same as FCFS).
- Quantum = 1: RR maximizes context switches, still correct.
- Quantum >= max burst: RR behaves like FCFS.

10. Discussion

10.1 Challenges Faced

- Round Robin queue management: ensuring newly arrived processes are enqueued after each quantum execution and not before, to respect arrival order.
- SJF tie-breaking: when two processes have the same burst time, the one with the earlier arrival time must be selected to ensure consistent, deterministic output.
- IDLE slot detection: correctly inserting idle slots when the CPU has no work to do requires checking if the current time is less than the next process's arrival.
- Gantt chart alignment: printing time ticks at the correct horizontal positions required careful width calculation for each cell.
- Preventing data corruption: since all three algorithms modify process data (completion times, remaining burst), a copyProcesses() function was essential to avoid contaminating the original process list.

10.2 Debugging Strategy

- Print debugging: added temporary printf statements to trace queue state, current time, and selected process at each scheduling step.
- Manual trace: worked through small examples (2-3 processes) by hand to verify the expected Gantt chart, then compared with program output.
- Step-by-step testing: tested each algorithm independently before integrating the menu system.
- Edge case testing: explicitly tested one-process input, all-same-burst, and zero-arrival-time scenarios.

- GCC warnings: compiled with -Wall flag to catch uninitialized variables and type mismatches early.

10.3 Observations

SJF consistently outperformed FCFS and RR on average waiting time for the tested inputs, confirming its theoretical optimality. However, process starvation was observable when long processes were repeatedly bypassed by shorter ones. Round Robin provided the fairest response times at the cost of higher average waiting and turnaround times, especially with smaller quanta.

11. Conclusion

This project successfully implemented a CPU Scheduling Simulator in C that demonstrates three fundamental scheduling algorithms: FCFS, SJF (Non-Preemptive), and Round Robin. The simulator generates ASCII Gantt charts and computes complete performance metrics, enabling direct comparison of algorithm behavior on the same process set.

Through this implementation, the following were learned:

- The direct relationship between scheduling decisions and system performance metrics.
- How non-preemptive algorithms differ structurally from preemptive ones in code.
- The importance of careful queue management in Round Robin to avoid incorrect ordering.
- How to visualize scheduling behavior using Gantt charts programmatically.
- The trade-offs inherent in each algorithm: simplicity vs. optimality vs. fairness.

The project reinforced that no single scheduling algorithm is universally optimal. FCFS is simple but inefficient; SJF minimizes average wait but risks starvation; Round Robin is fair but context-switch heavy. Modern OS schedulers combine techniques from all three to achieve practical performance goals.

12. Future Enhancements

- Preemptive SJF (SRTF): Implement Shortest Remaining Time First, where a running process is preempted if a shorter process arrives.
- Priority Scheduling: Add a priority field to each process and implement both preemptive and non-preemptive priority scheduling.
- Multilevel Queue Scheduling: Implement separate queues for different process types (interactive, batch, system) with inter-queue scheduling.
- Aging mechanism: Add aging to SJF to prevent starvation by gradually increasing the priority of waiting processes.
- Dynamic Quantum RR: Implement an adaptive time quantum that adjusts based on process burst time distribution.
- Graphical Interface: Port the simulator to a web-based interface (HTML/CSS/JS) with animated Gantt charts and real-time metric updates.
- File Input/Output: Support reading process data from CSV files and exporting results to text or CSV for further analysis.
- Multi-core Simulation: Extend to simulate scheduling on multiple CPUs with load balancing.
- Context Switch Overhead: Model context switch cost (e.g., 1 time unit per switch) to simulate real-world overhead in RR.
- Performance graphs: Plot average waiting time vs. time quantum for Round Robin to find the optimal quantum for a given workload.

13. References

34. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons.
35. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson Education.
36. Stallings, W. (2017). Operating Systems: Internals and Design Principles (9th ed.). Pearson.
37. Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard real-time environment. Journal of the ACM, 20(1), 46-61.
38. Suri, P. K., & Jolly, A. (2012). Design of dynamic time quantum based on intelligent round-robin scheduling algorithm. International Journal of Computer Science Issues, 9(4).
39. Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. Communications of the ACM, 17(7), 365-375.
40. GeeksforGeeks. (2024). CPU Scheduling Algorithms. Retrieved from https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/
41. TutorialsPoint. (2024). Operating System — Process Scheduling. Retrieved from https://www.tutorialspoint.com/operating_system/os_process_scheduling.htm
42. Linux Kernel Documentation. (2024). Completely Fair Scheduler (CFS). Retrieved from https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html